

# Docker Compose: Local Quick Start

---

By [Carlos Bazaga](#)

*With GPT-5.6 Luna as assistant writer.*

Use Docker Compose to build and run the service locally without manually exporting environment variables in each terminal session. Compose loads them from `.env` by default.

For extra protection, store `.env` in a separate folder outside the workspace, such as one level above the project. This keeps secrets out of version control and away from AI agents or other tools that scan the workspace.

## Name the Dockerfile Stage

Give the final image stage a name so Compose can target it explicitly.

In the `Dockerfile` change:

```
FROM python:3.13-slim
```

To:

```
FROM python:3.13-slim as backend
```

## Create `docker-compose.yaml` from Docker Commands

Create a `docker-compose.yaml` file and map the relevant parts of your existing Docker commands to its service.

Alternatively, copy the working example provided in this same folder and adapt it to your project.

- `docker build` arguments go under the service's `build` entry:

```
docker build \  
  --build-arg  
  NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY="$NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY" \  
  -t consultation-app .
```

Becomes:

```
build:  
  context: .                # Same period at the end of the command.  
  (Current folder)  
  provenance: false        # To avoid Lambda reject image as not OCI  
  compatible.  
  target: backend          # Dockerfile target image for multistage  
  ones.  
  args:                    # Same one as on the command line.  
    - NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=${NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY}  
  image: consultation-app   # Same as on the command line.
```

On Windows or macOS, if Lambda still rejects the image, try setting

`BUILDX_NO_DEFAULT_ATTESTATIONS=1` environment variable on your session before the build.

- `docker run` options go on the service itself:

```
docker run -p 8000:8000 \  
  -e CLERK_SECRET_KEY="$CLERK_SECRET_KEY" \  
  -e CLERK_JWKS_URL="$CLERK_JWKS_URL" \  
  -e OPENAI_API_KEY="$OPENAI_API_KEY" \  
  consultation-app
```

Becomes:

```
backend:  
  container_name: SaaS_Backend # Preferred name so you don't get random  
  ones.  
  build:  
    ... # See above.  
  image: consultation-app # Same as on the command line.  
  environment: # Same ones as on the command line.  
    - OPENAI_API_KEY=${OPENAI_API_KEY}  
    - CLERK_JWKS_URL=${CLERK_JWKS_URL}  
    - CLERK_SECRET_KEY=${CLERK_SECRET_KEY}  
  ports: # Same one as on the command line.  
    - "8000:8000"  
  restart: on-failure  
  platform: linux/amd64 # Ensure image creation for the right  
  architecture.
```

The `platform` setting is useful when you need to build or run for a specific architecture, such as `linux/amd64`, on an ARM-based Mac.

*It should work through emulation out of the box. Otherwise, well... maybe Macs are not as "fully compatible" as they claim. 😊*

## Build and Run Locally

Run this command from the folder containing your Compose file:

```
docker compose up -d --build
```

- `--build` builds the image if it does not exist or if changes require a rebuild, then starts the service. It is safe to include this flag every time.
- `-d` runs the service in the background. Omit it to keep the logs attached to your terminal.

When `.env` is outside the workspace, pass its path with `--env-file`.

The path can be relative to the directory where you run the command, for example `../.env`:

```
docker compose --env-file path_to_env_file/.env up -d --build
```

To stop and remove the service, run:

```
docker compose down
```